SMART GREENHOUSE CONTROLLER

A Simulated Closed-Loop Embedded Control System

Built Entirely in a Zero-Hardware Simulation Environment (Tinkercad)

Abdulatef

Final-Year BEng (Hons) Mechatronics Engineering

Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur

Independent Project — Embedded Systems / IoT

Tools: Tinkercad Circuits • Arduino C/C++ • Embedded Sensor & Actuator Control

Table of Contents

1. Executive Summary

This project presents the design, development, and testing of a Smart Greenhouse Controller — a fully autonomous, closed-loop embedded control system that monitors environmental conditions and actuates real-world outputs in response. The system was designed and validated entirely in a circuit-simulation environment, without any physical hardware, demonstrating the ability to plan, wire, program, debug, and document a complete embedded systems project end to end.

The controller reads temperature, ambient light, and soil moisture through three analog sensors, processes the data with noise-reducing averaging, and drives five independent outputs — a variable-speed cooling fan, an irrigation servo valve, a grow light, an RGB status indicator, and an audible alarm — according to a priority-based state machine. A 16x2 LCD and a serial data log provide real-time and historical visibility into system behaviour.

Beyond the core control loop, the project incorporates engineering practices typically expected in production-grade embedded systems: sensor signal averaging, proportional (PWM) actuation instead of simple on/off switching, time-aware (day/night) decision logic, structured debugging of logic-ordering defects, and a documented, repeatable test protocol used to verify every system state.

2. Project Objectives

The project was undertaken to design and validate a multi-sensor, multi-actuator embedded control system using industry-standard tools, while building practical skills in circuit design, embedded C/C++ programming, and systematic debugging. Specific objectives included:

- Design a closed-loop control system that senses environmental conditions and autonomously actuates corrective outputs.
- Implement and wire a complete sensor-to-actuator circuit (Arduino Uno-based) using correct voltage-divider and pull-up/pull-down principles.
- Develop embedded firmware in Arduino C/C++ implementing signal averaging, PWM-based proportional control, and a priority-ordered display/status state machine.
- Apply a systematic test-and-debug methodology to identify and resolve logic-ordering defects in the control software.
- Produce professional technical documentation suitable for a portfolio, LinkedIn, or a formal engineering report.

3. Tools & Technologies Used

Category	Tool / Technology	Purpose
Simulation Platform	Tinkercad Circuits (Autodesk)	Browser-based schematic capture and real-time circuit simulation — zero physical hardware required
Microcontroller	Arduino Uno R3 (simulated)	System controller running the embedded firmware

Category	Tool / Technology	Purpose
Programming Language	Arduino C/C++	Firmware implementing sensing, control logic, and I/O
Libraries	LiquidCrystal.h, Servo.h	LCD character display driver and servo actuator control
Interface	Arduino Serial Monitor	Real-time telemetry and data logging output
Documentation	Microsoft Word	Professional project report for portfolio and LinkedIn

4. System Overview & Component Architecture

The system follows a classic sense-decide-act embedded control architecture: three analog sensors feed the Arduino Uno, which evaluates threshold- and priority-based logic each control cycle and drives five output devices, while a 16x2 LCD and the serial console provide continuous human-readable feedback.

4.1 Component List

Component	Role in System
Arduino Uno R3	Central controller — reads sensors, executes control logic, drives outputs
TMP36 Temperature Sensor	Analog temperature sensing for thermal management
Photoresistor + 10 kΩ Resistor	Voltage-divider light sensing for grow-light and day/night logic
Potentiometer (soil moisture simulator)	Simulates a soil-moisture sensor for irrigation control
Potentiometer (LCD contrast)	Sets LCD V0 contrast voltage for readable text output
16x2 LCD Display	Human-machine interface — live readings and system status
DC Motor	Cooling fan — variable-speed (PWM) thermal response
Micro Servo Motor	Irrigation valve actuator
LED (Grow Light)	Compensates for low ambient light
RGB LED	Visual system-status indicator (green/amber/red)
Buzzer	Audible critical-temperature alarm

4.2 Control Flow

Each control cycle (approximately 1 second) follows the same sequence: (1) read and average all three sensors, (2) update running minimum/maximum statistics, (3) evaluate fan PWM output against temperature thresholds, (4) evaluate grow-light and day/night state, (5) evaluate irrigation logic against moisture and daylight state, (6) evaluate RGB/buzzer alert level, (7) resolve the single highest-priority status message for the LCD, and (8) write a formatted line to the serial log.

5. Circuit Design & Wiring

The circuit was built in four sequential phases to keep the wiring manageable and verifiable at each stage: power distribution, the LCD interface, the sensor bank, and finally the actuator/output bank.

5.1 Power Distribution

From	To	Purpose
Arduino 5V	Breadboard red (+) rail	Common 5V supply for all components
Arduino GND	Breadboard blue (−) rail	Common ground return

5.2 LCD Display (16x2, 4-bit mode)

LCD Pin	Connects To	Notes
VSS	GND rail	Ground
VDD	5V rail	Logic power
V0	Contrast potentiometer wiper	Sets character contrast
RS	Arduino D12	Register select
RW	GND rail	Permanently in write mode
E	Arduino D11	Enable strobe
D4–D7	Arduino D5, D4, D3, D2	4-bit data bus
A (backlight +)	5V rail via 220 Ω resistor	Backlight current-limited
K (backlight −)	GND rail	Backlight return

5.3 Sensor Bank

Sensor	Wiring	Principle
TMP36 (temperature)	Left → 5V, Center → A0, Right → GND	Linear analog voltage output, 10 mV/°C
Photoresistor (light)	Leg 1 → 5V; Leg 2 → A1 and → 10 kΩ resistor → GND	Voltage divider: resistance rises as light falls
Potentiometer (moisture)	Left → 5V, Wiper → A2, Right → GND	Simulated variable analog input

5.4 Actuator & Output Bank

Output Device	Wiring	Control Signal
DC Motor (fan)	Pin 1 → D9 (PWM), Pin 2 → GND	analogWrite() variable speed
Servo (water valve)	Signal → D10, Power → 5V, Ground → GND	Servo.write() angle command
Grow-light LED	Anode → 220 Ω → D8, Cathode → GND	digitalWrite() on/off
RGB LED (common cathode)	R → 220 Ω → D6, G → 220 Ω → D7, B → 220 Ω → D13, Cathode → GND	digitalWrite() per channel
Buzzer	Positive → A3 (digital), Negative → GND	tone() / noTone()

6. Software Implementation & Key Features

The firmware was developed iteratively, starting from a working baseline (v1.0) and evolving into a feature-complete, debugged release (v2.1) through structured testing. The final implementation includes the following engineering-grade features:

Feature	Implementation
Sensor averaging	Each sensor is sampled 5 times per cycle and averaged to suppress electrical noise and jitter.
Proportional fan control	Fan speed is mapped from temperature (28–40 °C → PWM 100–255) instead of simple on/off switching, for smoother and more efficient thermal response.
Day/night-aware irrigation	The irrigation valve only opens when both the soil is dry and ambient light indicates daytime; at night it enters a distinct 'waiting' state instead of watering.
Priority-based status system	A seven-state priority ladder (ALERT! → HOT! → COOLING → WATERING → WAIT 4 MORN → GROWING → OPTIMAL) resolves which single message the LCD displays each cycle.

Feature	Implementation
RGB + buzzer alerting	Three-tier visual/audible alerting: green (optimal), amber (warning), red with buzzer (critical).
Serial telemetry & data logging	Live tabulated sensor/actuator data every cycle, plus a rolling minimum/maximum statistics report every 10 seconds.

7. Testing & Validation

A repeatable, six-scenario test protocol was defined and executed to verify that every system state could be reliably triggered and correctly displayed. Each test fixed all three sensor inputs to known values and confirmed the expected actuator, LCD, and RGB/buzzer response.

Test	Input Conditions (Temp / Light / Moisture)	Verified Result
Baseline — OPTIMAL	22 °C / bright / 90% wet	RGB green; LCD "OPTIMAL"; fan speed 0
Proportional fan — COOLING	30–40 °C / bright / 90% wet	Fan PWM rises with temperature; LCD "COOLING" → "ALERT!" at 35 °C+
Irrigation — WATERING	22 °C / bright / 10% dry	Servo opens to 90°; LCD "WATERING"
Day/night logic — WAIT 4 MORN	22 °C / dark / 10% dry	Servo blinks (waiting); LCD "WAIT 4 MORN"; serial status "NIGHT-WAIT"
Grow light — GROWING	22 °C / dark / 90% wet	Grow-light LED on; LCD "GROWING"
Data logging	Any — run 10+ seconds	Serial console prints rolling min/max temperature and moisture report

8. Debugging & Problem Solving

Structured testing surfaced two logic-ordering defects that would not have been obvious from a code read-through alone, both resolved through targeted analysis of the priority ladder rather than trial and error:

8.1 Defect: "WAIT 4 MORN" state never displayed

Root cause: the display priority ladder evaluated the grow-light condition before the night-time irrigation-wait condition, so whenever it was dark the LCD always resolved to "GROWING" first and the night-wait branch was never reached.

Fix: reordered the priority ladder so the night-wait condition is evaluated before the grow-light condition, allowing the correct state to surface.

8.2 Defect: proportional fan speed changes were barely noticeable

Root cause: the PWM mapping spanned the full 25–40 °C range to 0–255, producing only a small output change (roughly 85/255) at the first noticeable temperature rise.

Fix: narrowed the active mapping range to 28–40 °C and raised the output floor to 100, producing a stronger, more visible speed change for the same temperature swing.

Both fixes were verified using the same six-scenario test protocol described above before the system was accepted as complete.

9. Results & Discussion

The completed controller correctly and repeatably reproduces all seven defined system states across the full range of simulated environmental conditions. Sensor averaging visibly reduced reading jitter in the serial log compared to the initial single-sample implementation, and proportional PWM fan control produced a smoothly scaling response rather than an abrupt on/off transition.

The day/night irrigation logic highlights a realistic engineering trade-off: watering during darkness increases evaporation loss and fungal-growth risk in real greenhouses, so the system intentionally defers irrigation until daylight is detected — a decision that models real-world horticultural practice rather than treating soil dryness as the only variable.

As a zero-hardware project, all validation was performed within the Tinkercad simulation environment. The next logical step toward a physical deployment would be bench-testing on real Arduino hardware with the same sensors, followed by calibration against real-world sensor tolerances and enclosure design for outdoor use.

10. Skills Demonstrated

- Embedded systems design — sensor-to-actuator closed-loop control architecture
- Circuit design & schematic capture — voltage dividers, pull-down resistors, multi-rail power distribution
- Embedded C/C++ programming — Arduino firmware development, PWM control, state-machine design
- Signal processing fundamentals — multi-sample averaging for noise reduction
- Systematic debugging — root-cause analysis of logic-ordering defects using a repeatable test protocol
- Technical documentation — structured, professional engineering reporting
- Simulation-based prototyping — full project lifecycle without physical hardware, using Tinkercad Circuits

11. Future Enhancements

- Migrate to physical Arduino hardware and calibrate against real sensor tolerances.
- Add a second servo-driven vent window for passive cooling.
- Introduce automatic threshold adjustment based on time-of-day or season.
- Design a 3D-printed enclosure (modelled in Tinkercad's 3D workspace) for outdoor deployment.
- Extend serial telemetry into a logged CSV file and visualize trends in MATLAB.

- Add wireless connectivity (e.g., ESP32) for remote monitoring and control.

12. Conclusion

This project demonstrates the complete engineering lifecycle of an embedded control system — requirements, circuit design, firmware development, systematic testing, debugging, and professional documentation — executed entirely within a browser-based simulation environment. The resulting Smart Greenhouse Controller reliably manages three sensor inputs and five actuator outputs through a well-structured, priority-based control system, reflecting practical skills directly transferable to real-world embedded systems and mechatronics engineering work.

Appendix A — Complete Firmware (Arduino C/C++, v2.1)

The final, debugged firmware controlling the Smart Greenhouse Controller is listed below in full.

```
#include <LiquidCrystal.h>
#include <Servo.h>

// ========== PIN DEFINITIONS ==========
const int tempPin = A0;
const int lightPin = A1;
const int moisturePin = A2;
const int fanPin = 9;
const int servoPin = 10;
const int growLightPin = 8;
const int redPin = 6;
const int greenPin = 7;
const int bluePin = 13;
const int buzzerPin = A3;

// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
Servo waterValve;

// ========== THRESHOLDS ==========
const float TEMP_HIGH = 28.0;
const float TEMP_CRITICAL = 35.0;
const int LIGHT_DARK = 300;
const int MOISTURE_DRY = 400;

// ========== DATA LOGGING ==========
float maxTemp = -100.0;
float minTemp = 1000.0;
float maxMoisture = 0;
float minMoisture = 1023;
unsigned long startTime = 0;
int logCounter = 0;

// ========== AVERAGING ==========
const int NUM_SAMPLES = 5;

float readAverageTemp() {
  long sum = 0;
  for (int i = 0; i < NUM_SAMPLES; i++) {
    sum += analogRead(tempPin);
    delay(5);
  }
  float avgReading = sum / (float)NUM_SAMPLES;
  float voltage = avgReading * (5.0 / 1023.0);
  return (voltage - 0.5) * 100.0;
}

int readAverageLight() {
  long sum = 0;
  for (int i = 0; i < NUM_SAMPLES; i++) {
    sum += analogRead(lightPin);
    delay(5);
  }
  return sum / NUM_SAMPLES;
}
```

```
int readAverageMoisture() {
  long sum = 0;
  for (int i = 0; i < NUM_SAMPLES; i++) {
    sum += analogRead(moisturePin);
    delay(5);
  }
  return sum / NUM_SAMPLES;
}

void setup() {
  Serial.begin(9600);

  pinMode(fanPin, OUTPUT);
  pinMode(growLightPin, OUTPUT);
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  waterValve.attach(servoPin);
  waterValve.write(0);

  lcd.begin(16, 2);
  lcd.print("Greenhouse Ready");
  delay(2000);
  lcd.clear();

  startTime = millis();

  Serial.println("======================================");
  Serial.println("   SMART GREENHOUSE CONTROLLER v2.1");
  Serial.println("======================================");
  Serial.println("Temp(C) | Light | Moisture | FanSpeed | Grow | Water | Status");
  Serial.println("-------------------------------------------------------------");
}

void loop() {
  // --- READ SENSORS ---
  float temperature = readAverageTemp();
  int lightLevel = readAverageLight();
  int moistureLevel = readAverageMoisture();

  // --- UPDATE MIN/MAX ---
  if (temperature > maxTemp) maxTemp = temperature;
  if (temperature < minTemp) minTemp = temperature;
  if (moistureLevel > maxMoisture) maxMoisture = moistureLevel;
  if (moistureLevel < minMoisture) minMoisture = moistureLevel;

  // --- SMOOTH FAN CONTROL (starts at 28C, stronger PWM range) ---
  int fanSpeed = 0;
  bool fanOn = false;
  if (temperature > TEMP_HIGH) {
    fanSpeed = map(constrain(temperature, 28, 40), 28, 40, 100, 255);
    analogWrite(fanPin, fanSpeed);
    fanOn = true;
  } else {
    analogWrite(fanPin, 0);
  }

  // --- GROW LIGHT ---
  bool lightOn = false;
```

```
if (lightLevel < LIGHT_DARK) {
  digitalWrite(growLightPin, HIGH);
  lightOn = true;
} else {
  digitalWrite(growLightPin, LOW);
}

// --- WATER VALVE (daytime only) ---
bool waterOn = false;
bool isDaytime = (lightLevel > LIGHT_DARK);

if (moistureLevel < MOISTURE_DRY && isDaytime) {
  waterValve.write(90);
  waterOn = true;
} else if (moistureLevel < MOISTURE_DRY && !isDaytime) {
  // Night + dry = blink to show "waiting for morning"
  int blinkPos = (millis() / 500) % 2 == 0 ? 15 : 0;
  waterValve.write(blinkPos);
} else {
  waterValve.write(0);
}

// --- RGB + BUZZER ---
bool critical = false;
if (temperature > TEMP_CRITICAL) {
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
  tone(buzzerPin, 1000, 500);
  critical = true;
} else if (temperature > TEMP_HIGH || moistureLevel < MOISTURE_DRY) {
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, HIGH);
  digitalWrite(bluePin, LOW);
  noTone(buzzerPin);
} else {
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, HIGH);
  digitalWrite(bluePin, LOW);
  noTone(buzzerPin);
}

// --- LCD DISPLAY (FIXED PRIORITY) ---
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(temperature, 1);
lcd.print(" L:");
lcd.print(lightLevel);

lcd.setCursor(0, 1);
lcd.print("M:");
lcd.print(moistureLevel);
lcd.print(" ");

// PRIORITY ORDER (highest to lowest):
if (critical) {
  lcd.print("ALERT!");
}
else if (fanOn && fanSpeed > 220) {
  lcd.print("HOT!");
}
```

```
  else if (fanOn) {
    lcd.print("COOLING");
  }
  else if (waterOn) {
    lcd.print("WATERING");
  }
  else if (!isDaytime && moistureLevel < MOISTURE_DRY) {
    // FIXED: Now checked BEFORE GROWING
    lcd.print("WAIT 4 MORN");
  }
  else if (lightOn) {
    lcd.print("GROWING");
  }
  else {
    lcd.print("OPTIMAL");
  }

  // --- SERIAL MONITOR ---
  Serial.print(temperature, 1);
  Serial.print("    | ");
  Serial.print(lightLevel);
  Serial.print("   | ");
  Serial.print(moistureLevel);
  Serial.print("      | ");
  Serial.print(fanSpeed);
  Serial.print("      | ");
  Serial.print(lightOn ? "ON " : "OFF");
  Serial.print(" | ");
  Serial.print(waterOn ? "ON " : (moistureLevel < MOISTURE_DRY ? "WAIT" : "OFF"));
  Serial.print(" | ");

  if (critical) Serial.println("CRITICAL");
  else if (!isDaytime && moistureLevel < MOISTURE_DRY) Serial.println("NIGHT-WAIT");
  else if (fanOn || waterOn || lightOn) Serial.println("ACTIVE");
  else Serial.println("OK");

  // --- DATA LOG (every 10 seconds) ---
  logCounter++;
  if (logCounter >= 10) {
    logCounter = 0;
    unsigned long elapsed = (millis() - startTime) / 1000;

    Serial.println();
    Serial.println("----- DATA LOG (" + String(elapsed) + "s) -----");
    Serial.print("Temp:  Min="); Serial.print(minTemp, 1);
    Serial.print("  Max="); Serial.println(maxTemp, 1);
    Serial.print("Moist: Min="); Serial.print(minMoisture);
    Serial.print("  Max="); Serial.println(maxMoisture);
    Serial.println("-------------------------------");
    Serial.println();
  }

  delay(1000);
}
```